Reflected XSS into a JavaScript String with Single Quote and Backslash Escaped

1. Overview

- **Vulnerability:** Reflected Cross-Site Scripting (XSS)
- **Category:** Client-Side Injection
- **Affected Context:** JavaScript string
- **Severity:** Context-dependent
- **Lab Platform:** PortSwigger Web Security Academy
- **Status:** Educational lab write-up

2. Description

This vulnerability occurs when user-controlled input is reflected into a JavaScript string within an HTML document. Although the application attempts to escape single quotes and backslashes, an incomplete or incorrectly ordered escaping mechanism may still allow an attacker to break out of the intended string context.

If the injected input is interpreted as executable JavaScript, an attacker may execute code in the context of a victim's browser.

3. Root Cause

The underlying issue is insufficient context-aware output encoding. JavaScript string escaping alone may not be sufficient when data is embedded within an HTML script context. The effectiveness of the escaping depends on how the application processes the input and constructs the final response.

4. Proof of Concept (PoC)

Testing environment: Authorized PortSwigger Web Security Academy lab.

1. Open the assigned lab in a browser.
2. Identify the input parameter reflected into a JavaScript string.
3. Submit a harmless test value and inspect the resulting HTML source.
4. Examine how single quotes, backslashes, and other special characters are encoded.

5. In the authorized lab, test whether the input can escape the JavaScript string and execute a harmless proof-of-execution statement.
6. Confirm the result using browser developer tools.

Example test concept:

```
None
'; alert(1); //
```

This is a conceptual payload only. Whether it works depends on the actual JavaScript context and the application's escaping behavior. If single quotes and backslashes are escaped correctly, this exact payload may not execute.

5. Security Impact

If successfully exploited, reflected XSS may allow an attacker to:

- Execute JavaScript in the vulnerable application's origin.
- Modify page content or perform actions through the victim's authenticated session.
- Access sensitive information exposed to client-side scripts.
- Display deceptive content or redirect users to malicious websites.

The actual impact depends on the application's functionality, browser protections, and available security controls.

6. Recommended Remediation

- Avoid inserting untrusted input directly into executable JavaScript.
- Prefer passing data through safely serialized JSON or non-executable data attributes.
- Apply context-aware output encoding appropriate to the exact HTML and JavaScript context.
- Use well-maintained templating libraries with automatic contextual escaping.
- Validate input as a supplementary measure, not as the primary XSS defense.
- Deploy a restrictive Content Security Policy (CSP) as an additional security layer.
- Add regression tests for quotes, backslashes, line terminators, and HTML/script parsing edge cases.

7. Verification

After remediation:

1. Repeat the original test cases.

2. Confirm that input remains data rather than executable code.
3. Test relevant special characters and encoding variations.
4. Verify that legitimate application functionality remains intact.

8. References

- PortSwigger: Cross-Site Scripting (XSS)
- OWASP: Cross Site Scripting Prevention Cheat Sheet

9. Disclaimer

This write-up is intended for authorized security testing and educational purposes. Testing should only be performed against systems for which explicit permission has been granted.